Assignment 1: Analysis of SVD for Fine-Tuning LLMs

蕭至恆 (Student ID 413551034) · Big Data Analytics Techniques and Applications, Fall 2026

GitHub: https://github.com/CH-Joshua-Hsiao/NYCU-BigDataAnalytics/tree/main/Assignment1

Setup: Llama-3.2-1B fine-tuned with PiSSA on 25,000 MetaMathQA samples (1 epoch, lr 2e-5, global batch 128, FP16, all 7 projection types), evaluated on 1,000 GSM8K and 1,000 MATH test problems. All runs were done on a single local RTX 3090 (24 GB) with micro-batch 4 × 32 accumulation steps; SVD initialization ran on the CPU.

Question 1: Accuracy of the original and fine-tuned models

Accuracy	Original LLM	Fine-Tuned (RANK = 16)	Fine-Tuned (RANK = 64)	Fine-Tuned (RANK = 256)
GSM8K	3.4%	15.6%	20.7%	27.3%
MATH	0.1%	3.7%	4.8%	5.4%

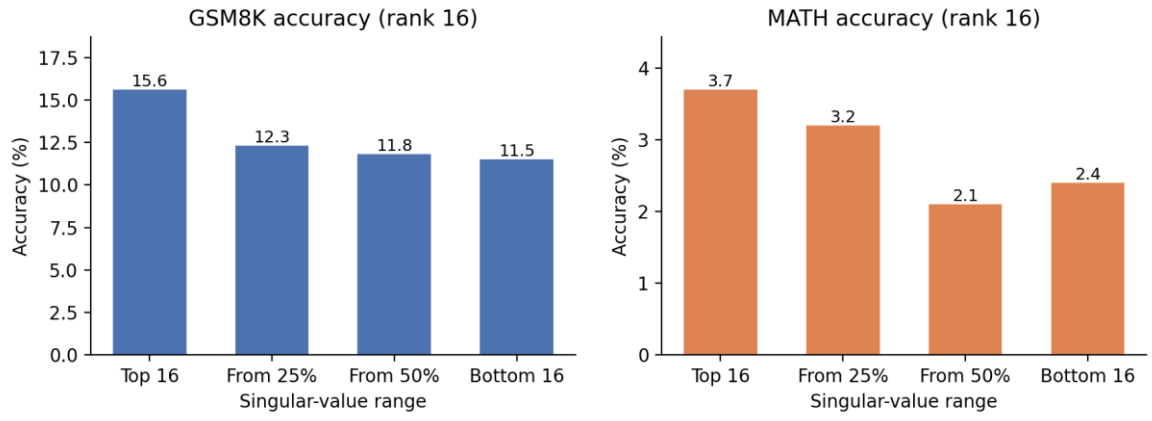
- **Baseline.** The original model scores only 3.4% on GSM8K and 0.1% on MATH. It is a base (non-instruction) model, so it rarely produces a final answer in a parseable form: it never writes "The answer is:", and 959 of the 1,000 MATH outputs contain no extractable answer at all. Fine-tuning on MetaMathQA teaches both step-by-step reasoning and the answer format, so even rank 16 jumps to 15.6% / 3.7%.
- **Effect of rank.** Accuracy rises monotonically with rank: GSM8K 15.6% → 20.7% → 27.3%, MATH 3.7% → 4.8% → 5.4%. A larger rank gives more trainable parameters (0.9% → 3.6% → 14.6% of the model) and, with PiSSA, the adapter starts from a larger share of each weight's principal components, so the same learning rate and one epoch move a bigger, more important subspace. The final training loss drops accordingly (0.455 → 0.414 → 0.375).
- **Dataset difference.** GSM8K gains far more (+11.7 points from rank 16 to 256) than MATH (+1.7). GSM8K problems are short arithmetic word problems that closely match the MetaMathQA augmentations, and its gains have not saturated at rank 256. MATH needs competition-level knowledge and LaTeX-style answers (fractions, radicals, intervals) that a 1B model cannot acquire from 25k examples in one epoch; about 45% of MATH outputs still contain no parseable answer, so its curve flattens much sooner.

Question 2: Training time

	Fine-Tuned (RANK = 16)	Fine-Tuned (RANK = 64)	Fine-Tuned (RANK = 256)
Training Time (min.)	36.0	38.8	64.0
Trainable Parameters	11.3 M	45.1 M	180.4 M

- **Trend.** Going from rank 16 to 64 multiplies trainable parameters by 4× but adds only 2.7 minutes (+8%); going to rank 256 (16× parameters) takes 64.0 minutes (+78% over rank 16). Training time therefore grows with rank, but far slower than the parameter count.
- **Explanation.** Each step is dominated by the forward and backward pass through the frozen 1.2B-parameter residual weights and by attention/activation work, which do not depend on the rank. The adapter adds two thin matmuls costing r(d+k) per token versus dk for the base weight: for a 2048×2048 projection that is about 1.6% extra at r = 16 but 25% at r = 256 (and its weight gradients must also be computed, unlike the frozen base). Rank 256 also has 180M trainable parameters with FP32 AdamW states and gradients, adding optimizer-step time and memory traffic. So the cost is nearly flat at small ranks and becomes noticeable only when r is a sizeable fraction of the hidden size. The one-off SVD initialization (~100 s on the CPU) is excluded and is the same for every rank, since a full SVD is computed regardless of r.

Question 3: Fine-tuning different singular-value ranges (rank 16)

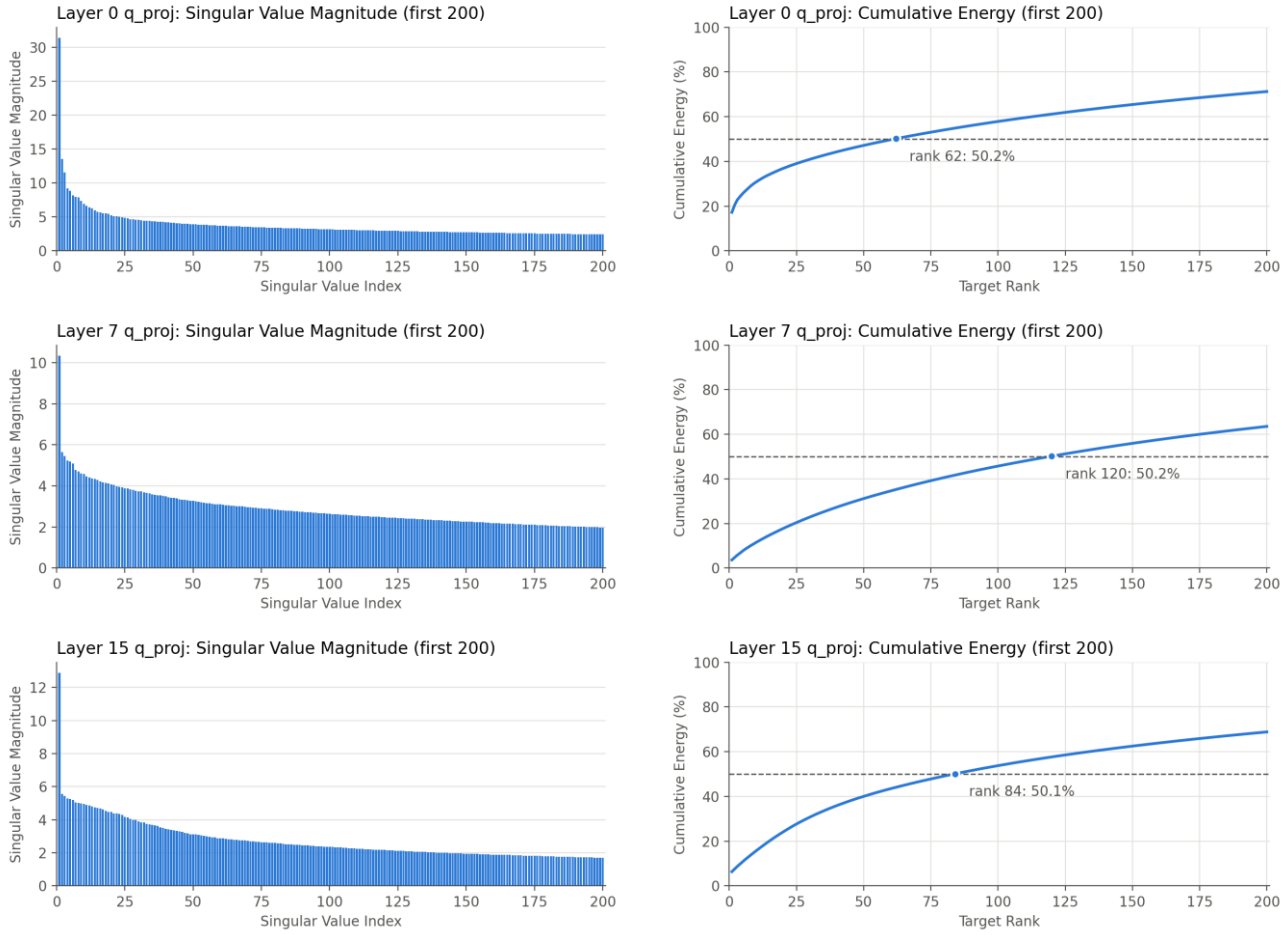


Range	Top 16	From 25%	From 50%	Bottom 16
GSM8K	15.6%	12.3%	11.8%	11.5%
MATH	3.7%	3.2%	2.1%	2.4%

- **Trend.** The top-16 (principal) range is clearly best on both datasets (15.6% GSM8K, 3.7% MATH). Moving the 16 directions down the spectrum lowers accuracy: 12.3% / 11.8% / 11.5% on GSM8K and 3.2% / 2.1% / 2.4% on MATH for the 25%, 50% and bottom ranges. The training loss shows the same order (average of the last 10 updates: 0.419 top, 0.462 25%, 0.467 50%, 0.476 bottom). The non-principal ranges differ from each other by at most ~1 point, which is within the sampling noise of a 1,000-question test set (standard error ≈ 1.0 point on GSM8K, ≈ 0.5 on MATH), so they are essentially tied; only the gap to the top range is clear.
- **Why the principal range wins.** With PiSSA the adapter is initialized as $A = U_r\sqrt{S_r}$, $B = \sqrt{S_r}\, V_r^T$, so the trainable part is exactly the selected slice of the weight and fine-tuning changes the model only inside that subspace. The top singular directions carry the most energy (Q4: the top 16 hold 15-35% of a q_proj's energy, a 16-wide band at the 25% or 50% position only ~0.1-0.7%) and correspond to the input/output directions the layer actually uses, so updating them changes the layer's behaviour most efficiently. The adapter factors are also larger, which makes their gradients ($\partial L/\partial A \propto B$, $\partial L/\partial B \propto A$) larger and learning faster. Middle and bottom directions are small, nearly unused by the pretrained model, so a rank-16 update there is a weaker lever.
- **Why the other ranges still learn.** Even the bottom range ($\sigma \approx 0$) improves far beyond the original model (3.4% → 11.5% GSM8K). AdamW normalizes each update by the gradient's running magnitude, so tiny factors still move by about the learning rate per step, and any 16-dimensional subspace is enough to learn the MetaMathQA answer format and some reasoning patterns. The principal subspace adds a further +3-4 points on GSM8K by also adapting the directions that matter most, consistent with the PiSSA paper's claim that tuning principal components converges faster and generalizes better than tuning the minor ones.

Question 4: SVD of q_proj in the original model (layers 0, 7, 15)

	Layer 0	Layer 7	Layer 15
Minimum Target Rank (≥ 50% energy)	62	120	84
σ1 / σ16	31.4 / 5.7	10.3 / 4.2	12.9 / 4.7
Energy in top 16 / top 200	34.8% / 71.2%	15.4% / 63.5%	21.0% / 68.8%



Left: singular value magnitudes (bar charts). Right: cumulative energy (line charts). First 200 of 2,048 singular values; energy = $\Sigma\sigma_i^2 / \Sigma\sigma^2$, dashed line = 50%.

- **Shape.** Every q_proj spectrum has a sharp head and a long, slowly decaying tail: a few large singular values (σ1 is 2-5× σ16) followed by an almost flat plateau. Reaching 50% energy needs only 62-120 of 2,048 directions (3-6% of full

rank), but even the first 200 directions hold just 64-71% of the energy, so the weights are not truly low-rank; the remaining energy is spread thinly over hundreds of directions.

- **Layer differences.** Layer 0 is the most concentrated ($\sigma_1 = 31.4$ alone dominates; the top 16 hold 34.8% of the energy), layer 7 is the flattest (15.4% in the top 16, rank 120 needed) and layer 15 lies in between (rank 84). The first layer works directly on token embeddings, where a few dominant directions (e.g. common-token and position-related features) carry most of the signal; middle layers mix many distributed features, so their queries use more directions; the last layers specialize again toward next-token prediction.
- **Implication.** A fixed adapter rank therefore captures a very different share of each layer: rank 16 covers about 35% of layer 0's energy but only 15% of layer 7's. This supports PiSSA's idea of adapting the principal subspace, and suggests that layer-wise (adaptive) rank allocation would use a parameter budget more evenly.